GPU-Assisted AES Acceleration for Bulk Data Processing in Telecommunications: Experimental Evidence and Practical Constraints

Marko Mihajlovic[1]

Abstract:

High-throughput cryptographic processing is an important requirement in modern telecommunications infrastructure, where GPU acceleration offers opportunities for improving bulk data encryption. This study investigates the conditions under which GPU-assisted AES-128 processing can provide measurable benefits over hardware-accelerated CPU encryption. A validation-first experimental approach evaluates AES in CTR and ECB modes across three heterogeneous computing configurations, using paired workloads, five measurement sessions per system, and separate device-resident and synchronous transfer-inclusive GPU execution boundaries. For 32 MiB AES-CTR workloads, the GPU processing pipeline achieves median paired speedups of 1.354× on the T600 Laptop GPU and 4.761× on the Tesla T4 relative to their respective AES-NI CPU implementations, with improvements observed in all five sessions. In contrast, the GTX configuration achieves only 0.935×, demonstrating that acceleration is not universally beneficial. The results establish experimentally supported conditions under which GPU processing can improve bulk AES throughput despite synchronous data transfers. These findings provide experimental evidence that GPU-assisted AES-CTR processing can improve bulk encryption throughput over hardware-accelerated CPU execution, even when synchronous data transfers are included. The demonstrated gains identify concrete opportunities for accelerating cryptographic workloads in heterogeneous telecommunications systems, although improvements in complete IPsec and authenticated network processing remain to be established.

Keywords: Advanced Encryption Standard (AES), AES-NI, GPU acceleration, CUDA, heterogeneous computing, cryptographic benchmarking, data-transfer overhead, reproducible performance evaluation.

1 Introduction

The increasing volume of data processed by modern telecommunications infrastructure creates a continuing demand for efficient cryptographic operations. Communication gateways, network services, distributed processing systems, and data-intensive applications must protect information while maintaining adequate throughput and acceptable processing latency. The Advanced Encryption Stan-

[1]Singidunum University, Belgrade, Serbia; ORCID: 0009-0006-7979-0341

dard (AES) remains a fundamental building block of symmetric encryption, with its algorithmic specification defined by NIST FIPS 197 [1]. However, the performance of AES in practical systems depends not only on the cryptographic algorithm but also on the available hardware acceleration, execution strategy, and movement of data between processing resources.

Modern processors provide dedicated AES instructions, commonly known as AES-NI, which substantially accelerate the execution of AES transformations [2]. Graphics processing units (GPUs), on the other hand, offer a highly parallel execution environment capable of processing large numbers of independent computational tasks. This capability is particularly relevant to the Counter (CTR) mode of AES, where independent counter blocks can be encrypted in parallel to generate the required keystream [3]. For telecommunications-oriented workloads involving large data buffers or collections of independent data units, GPU acceleration may therefore provide an additional processing option.

The potential benefit is especially interesting for systems that already contain general-purpose GPUs but rely primarily on the host CPU for cryptographic operations. Such systems could potentially employ existing heterogeneous computing resources to increase bulk encryption throughput without requiring a dedicated cryptographic accelerator. However, transferring an encryption workload to a GPU introduces costs associated with host-to-device data movement, kernel execution, synchronization, and returning the processed data to host memory. These operations can reduce or eliminate the computational advantage of parallel execution. Consequently, high device-level AES throughput does not necessarily imply improved performance for the complete processing path [4, 5].

Previous research has established the feasibility of GPU-accelerated AES and demonstrated substantial improvements through parallel processing, memory organization, and architecture-specific optimization [6–10]. Earlier studies have also investigated the overlapping of host–device transfers with computation, demonstrating that communication overhead is an established concern in GPU cryptographic processing [8]. Nevertheless, the throughput achieved by highly optimized kernels does not, by itself, determine whether GPU offloading is beneficial for a particular system, especially when the host processor already provides hardware-assisted AES execution.

For telecommunications infrastructure, the relevant problem is therefore not simply how fast AES can execute on a GPU, but under which operating conditions GPU-assisted processing can improve the throughput of a cryptographic workload while accounting for the costs of transferring data through the accelerator. This distinction becomes important when considering different processing strategies, such as encrypting data directly on the CPU, processing large buffers on a GPU, or organizing independent workloads into batches suitable for parallel execution.

Although batching and device-resident processing offer potential architectural benefits, their effectiveness depends on workload size, data placement, execution overhead, and application-level latency requirements.

The central research question addressed in this study is whether GPU-assisted AES processing can provide a reproducible improvement in bulk encryption performance over hardware-accelerated CPU execution, and which workload, platform, and data-movement conditions determine the applicability of that improvement. Particular attention is given to distinguishing improvements obtained from isolated GPU computation from those that remain measurable when the processing path includes synchronous host–device transfers and synchronization.

To investigate this question, the study evaluates validated AES-128 implementations in Electronic Codebook (ECB) and Counter (CTR) modes across three heterogeneous computing configurations. ECB is included as a controlled reference for independent block processing, while CTR represents the primary parallelizable encryption workload of interest. The experimental protocol combines cryptographic correctness verification, controlled inputs, paired execution conditions, repeated measurement sessions, and explicitly defined timing boundaries. Hardware-assisted CPU AES-NI serves as the reference for determining whether GPU execution provides an additional performance benefit.

The study contributes experimentally verified evidence about the conditions under which GPU-assisted bulk AES processing can be advantageous, together with a reproducible methodology for identifying the limits of that advantage. Rather than proposing a new AES algorithm or claiming a universally superior implementation, the contribution lies in establishing measurable operating conditions under which existing GPU resources can improve encryption throughput despite the additional costs of data movement. Equally important, the study identifies configurations in which these costs or host implementation characteristics prevent a consistent improvement.

The significance of these findings extends to the design of heterogeneous telecommunications processing architectures. Large-volume encryption, suitable workload aggregation, and data-resident processing represent potential areas in which GPU resources could complement CPU cryptographic instructions. However, improved AES-CTR buffer processing must not be interpreted as demonstrated acceleration of complete network security protocols. For example, AES-GCM incorporates authentication processing in addition to counter-mode encryption [11], and practical protocol implementations introduce further costs associated with packet handling, cryptographic state, and data delivery. These requirements must be evaluated separately before extending the findings to complete telecommunications applications.

The remainder of the paper develops this argument systematically. Section 2 examines existing GPU AES acceleration techniques and the methodological limitations of reported performance results. Section 3 describes the experimental design, validation procedures, execution boundaries, and statistical analysis. Section 4 presents the measured operating conditions and observed acceleration outcomes. Section 5 considers their implications for telecommunications-oriented processing, practical constraints, and future applications. Finally, Section 6 summarizes the principal findings and their significance for GPU-assisted AES acceleration.

2 Related Work and Research Gap

2.1 GPU-Based AES Acceleration

The use of graphics processing units for accelerating AES has been investigated extensively, with research demonstrating that parallel execution can substantially increase encryption throughput for suitable workloads. Early contributions by Manavski [6] and Harrison and Waldron [7] established the feasibility of executing AES on programmable GPU architectures.

The potential for parallel execution depends partly on the cryptographic mode. ECB permits independent block transformations, while CTR permits the parallel generation of keystream blocks using distinct counter values [3]. This property is particularly relevant to bulk encryption, where sufficiently large amounts of independent work can be distributed across GPU execution threads. Nevertheless, ECB remains unsuitable for protecting general structured data and is primarily useful here as a controlled performance reference.

Subsequent research focused on improving the efficiency of GPU AES implementations. Iwai et al. [8] examined thread granularity, memory organization, and the interaction between computation and data transfers. Their CUDA implementation achieved approximately 35 Gbit/s for device-side AES processing and reported approximately 22.5 Gbit/s when transfer-aware execution was considered. These findings established that GPU encryption performance depends not only on computational parallelism but also on how data are transferred and organized. Wang and Chu [9] investigated lookup-table-based AES implementations and GPU execution strategies, while Tezcan [10] demonstrated the performance benefits of shared-memory optimization and avoiding bank conflicts. The latter reported AES-128 encryption throughput of 878.6 Gbit/s on an NVIDIA RTX 2070 Super.

More recent bit-sliced implementations have further increased GPU AES throughput. A study published in 2024 [12] reported high-performance AES-CTR

and AES-ECB execution on desktop and embedded NVIDIA platforms. These developments illustrate the considerable computational potential of architecture-specific optimization and parallel cipher representations.

However, these performance achievements do not necessarily establish the throughput available to a telecommunications application. Device-side encryption rates can be substantially different from the performance of a processing pipeline that must receive data, execute the cryptographic operation, and deliver the resulting output. Consequently high GPU AES throughput provides evidence of computational capability, but additional information is needed to determine whether that capability can improve the processing of bulk communication data under practical execution constraints.

2.2 Hardware-Assisted CPU Encryption and Data Movement

Modern CPUs provide dedicated cryptographic acceleration through instructions such as AES-NI, which reduce the execution cost of AES transformations [2]. These capabilities establish a demanding performance reference for applications considering GPU offloading.

Lee et al. [13] demonstrated that conclusions about heterogeneous acceleration depend strongly on the optimization level of the implementations being evaluated. Their findings underline the importance of using an appropriately accelerated CPU reference rather than assuming that the performance of conventional software encryption represents the capability of the host processor.

For bulk cryptographic processing, the costs of moving data between processors are also important. Gregg and Hazelwood [4] established that meaningful CPU–GPU performance evaluation requires explicit consideration of data location and movement. A computational advantage obtained on data already resident in GPU memory may be reduced when input and output must cross the host–device boundary. CUDA programming and performance guidelines similarly emphasize minimizing unnecessary transfers, using appropriate synchronization, and considering overlap between data movement and computation [5, 14]. AES-specific research has already investigated these techniques [8].

These findings are relevant to communication infrastructure because cryptographic processing rarely occurs in isolation. A network-oriented system may receive data through a host-managed interface, organize it into processing buffers, execute encryption, and subsequently forward the protected output. The placement of cryptographic computation within this processing path influences whether acceleration is beneficial. For example, the usefulness of GPU processing may differ between a large contiguous buffer, independently aggregated messages, and individual latency-sensitive packets.

The present study therefore distinguishes GPU execution with data already resident in device memory from synchronous prepared-buffer processing that includes host-to-device and device-to-host transfers. The latter represents a broader, transfer-inclusive operation, although it does not include complete network input/output or application processing.

2.3 Relevance to Telecommunications Security

Telecommunications systems require cryptographic processing that balances throughput with latency, integrity protection, and correct management of cryptographic state. These requirements make the selection of an acceleration strategy more complex than maximizing the throughput of an isolated encryption kernel.

AES-CTR is particularly relevant to this discussion because counter-mode processing permits independent block-level encryption [3]. This property suggests opportunities for parallel processing of large buffers or suitably organized collections of independent data units.

However, confidentiality-only encryption is insufficient for many secure communication applications. Authenticated-encryption constructions such as AES-GCM combine counter-mode encryption with authentication through GHASH [11]. This distinction is important when considering protocols such as IPsec or TLS. An improvement in AES-CTR computation may benefit one component of an authenticated-encryption pipeline, but does not establish equivalent improvement in complete protocol processing.

Practical implementations must additionally account for authentication, nonce management, packet preparation, synchronization, and output delivery. Furthermore, aggregating multiple small messages may improve the amount of work available for parallel execution while introducing batching and queueing latency.

These considerations suggest that GPU acceleration is particularly worth investigating for telecommunications workloads involving substantial volumes of independent cryptographic processing. Possible application contexts include bulk data protection, traffic aggregation, and cryptographic operations within heterogeneous communication gateways and data-processing services. Nevertheless, the effectiveness of these architectural approaches cannot be inferred solely from peak GPU throughput. It must be assessed against the available hardware acceleration and the operational requirements of the intended processing path.

The present investigation addresses this prerequisite through validated measurements of bulk AES processing. It does not claim to implement or benchmark a complete telecommunications security protocol.

2.4 Experimental Reliability and Reproducibility

Reliable evidence of performance improvement requires more than reporting the highest measured throughput.

Mytkowicz et al. [15] demonstrated that changes in experimental conditions can introduce systematic bias and substantially alter performance results. Kalibera and Jones [16] emphasized the importance of accounting for variability across multiple levels of experimental replication, while Hoefler and Belli [17] discussed scientifically defensible performance evaluation of parallel computing systems.

These contributions establish a methodological foundation for determining whether an observed acceleration is repeatable under the tested conditions. For cryptographic processing, functional correctness must also be established before performance measurements are accepted. High throughput provides little useful evidence if the implementation does not produce the expected cryptographic output.

The present study applies these principles through cryptographic validation, paired workloads, repeated study sessions, documented execution boundaries, and preservation of raw observations and experimental provenance. This approach supports the distinction between an isolated favorable measurement and a performance advantage observed consistently across multiple executions. It also allows results that favor CPU processing to be retained alongside cases where GPU acceleration provides an advantage.

2.5 Research Gap and Positioning of the Present Study

Previous research has demonstrated that AES can achieve substantial throughput on GPUs and that optimized implementations can exploit parallelism, memory organization, and specialized execution strategies [8, 10, 12]. The importance of data-transfer overhead, strong CPU references, and rigorous performance measurement is likewise well established [4, 13, 16]. The remaining problem is not whether AES can be accelerated on a GPU, but how to identify operating conditions under which that acceleration produces a repeatable benefit for bulk cryptographic processing when the host already supports hardware-assisted encryption and data movement cannot be ignored.

This question is particularly relevant to telecommunications-oriented systems, where the processing location and organization of encryption workloads may influence achievable throughput and latency. Existing high-throughput kernel results do not, by themselves, establish whether a particular GPU configuration is suitable for such processing. Conversely, an unfavorable transfer-inclusive result on one system does not demonstrate that GPU offloading is ineffective under different workload or platform conditions.

The present study addresses this practical evidence gap by evaluating AES-128 processing across three heterogeneous computing configurations using a common validation and measurement protocol. The evaluation considers ECB and CTR workloads of different sizes, with hardware-assisted CPU AES-NI serving as the reference for determining whether GPU processing offers an additional benefit. A distinction is maintained between device-resident execution and a synchronous prepared-buffer pipeline that includes the measured host–device transfers. Session-level measurements are preserved to determine whether observed improvements remain consistent across repeated executions.

The principal contribution is the identification of experimentally supported conditions in which GPU-assisted bulk AES processing provides an advantage, together with evidence of the constraints under which that advantage disappears or becomes inconsistent. Rather than proposing a new cryptographic primitive or claiming state-of-the-art GPU throughput, the study provides an empirical basis for assessing the feasibility of GPU-assisted AES processing in heterogeneous telecommunications architectures.

Its findings can inform the design of future bulk encryption and workload-placement strategies, while the performance of complete authenticated-encryption and network processing pipelines remains a separate application-level question.

3 Methodology

The experimental methodology is designed to determine under which operating conditions GPU-assisted AES processing can improve bulk encryption throughput while accounting for the overhead associated with accelerator execution. The evaluation focuses on AES-128 encryption, using hardware-assisted CPU AES-NI as the reference and a validated CUDA implementation as the alternative processing path.

The methodology follows a validation-first approach that combines cryptographic correctness checks, controlled input workloads, explicitly defined timing boundaries, repeated measurements, and independent verification of the resulting data. Its purpose is to identify reproducible acceleration conditions rather than to establish the maximum theoretical throughput of either processor architecture.

The experimental workloads represent contiguous encryption buffers of different sizes. They are intended to characterize the computational behavior relevant to bulk data processing, not to reproduce a complete telecommunications protocol or network packet-processing pipeline.

3.1 Experimental Platforms and Implementations

Three heterogeneous computing configurations were selected to investigate whether GPU-assisted AES processing can provide measurable benefits across different hardware and software environments.

The configurations include two laptop systems equipped with NVIDIA GPUs and one Linux-based cloud environment using an NVIDIA Tesla T4. Their characteristics are summarized in Table 1.

Table 1
Hardware and software configurations used for AES acceleration experiments.

ID	Hardware and environment	Software stack
GTX	Intel Core i5-10300H; GeForce GTX 1650 Ti, 4 GB; Windows 10	Driver 610.62; CUDA runtime 12.9; CuPy 13.6.0; PyCryptodome 3.23.0
T600	Intel Core i7-11850H; T600 Laptop GPU, 4 GB; Windows 11	Driver 581.95; CUDA runtime 12.9; CuPy 14.2.0; PyCryptodome 3.24.1
T4	Intel Xeon at 2.00 GHz, two exposed logical processors; Tesla T4; Linux/Google Colab	Driver 580.82.07; CUDA runtime 13.0; CuPy 14.0.1; PyCryptodome 4.0.0

The two laptop configurations represent accessible GPU-equipped computing systems, while the T4 provides a datacenter-oriented environment. The exact model of the cloud host CPU was not available in the archived environment records. Each configuration is treated as a complete hardware–software system. Differences in observed performance cannot be attributed solely to the GPU, operating system, processor generation, driver, or cryptographic library because these characteristics were not independently controlled.

The CPU reference uses PyCryptodome with hardware-assisted AES-NI explicitly enabled [2, 18]. Native backend initialization was checked for both evaluated encryption modes. This supports the intended use of AES-NI, although it does not constitute a direct trace of executed processor instructions. The GPU implementation is the validated CUDA kernel archived as `cuda_baseline`, executed through CuPy's custom kernel interface [19]. The kernel assigns one 16-byte AES block to each CUDA thread, with 128 threads per CUDA block.

This organization allows independent AES blocks to be distributed across GPU execution threads. It provides a controlled implementation for evaluating bulk encryption, but is not claimed to represent the highest attainable CUDA AES throughput. The selected CPU AES-NI and CUDA execution paths originate from a broader experimental campaign that also evaluated additional implementations. The present analysis retains only the paths relevant to the stated research objective.

3.2 Cryptographic Validation and Workload Design

Functional correctness is established before performance measurements are accepted for analysis. This requirement ensures that observed throughput corresponds to a valid implementation of the intended AES transformation.

Validation includes known-answer tests and consistency checks against CPU reference operations, using the AES specification and the definitions of the evaluated encryption modes [1, 3]. The archived validation records document successful CUDA execution and agreement with the expected cryptographic outputs. A failure of the required checks prevents the corresponding execution from being accepted as a valid performance result.

Correctness validation is complemented by verification of experimental configuration, completeness of measurement records, and source-code identity. Passing cryptographic test vectors does not independently establish that a timing method is free from measurement bias.The experiment evaluates AES-128 encryption in Electronic Codebook (ECB) and Counter (CTR) modes.

CTR is the principal mode of interest because independent counter blocks can be encrypted in parallel, providing a computational structure suitable for GPU-assisted bulk processing [3]. ECB is retained as a controlled reference for independent AES block transformations. Its inclusion supports analysis of mode-dependent implementation behavior and must not be interpreted as a recommendation to use ECB for general application data.

Five input sizes were evaluated:

$$N \in \{1\ \text{KiB},\ 16\ \text{KiB},\ 1\ \text{MiB},\ 16\ \text{MiB},\ 32\ \text{MiB}\}. \tag{1}$$

The selected sizes span small encryption requests and larger contiguous buffers suitable for examining bulk-processing throughput.

These workloads are not equivalent to individual telecommunications packets. In particular, processing a 32 MiB buffer does not demonstrate that independently received network packets can be grouped and encrypted with the same throughput or latency. Corresponding CPU and GPU executions use deterministic input generation and matching cryptographic parameters, including plaintext, encryption key, and mode-specific values.

The experiments use five study sessions per platform with random seeds 2003–2007. Each measured configuration includes five warm-up executions followed by twenty recorded timing repetitions. Warm-up operations are excluded from the reported timing records.

The selected analysis contains thirty measured configuration groups per session, with twenty observations per group. Consequently, each session contributes

600 observations, giving

$$3 \times 5 \times 30 \times 20 = 9\,000 \tag{2}$$

raw timing observations across the three systems.

Workload sizes are executed in ascending order, while mode and backend/measurement-scope jobs are shuffled according to the session seed. Benchmark jobs execute sequentially rather than concurrently.

The retained observations preserve their original execution context, including interleaved jobs excluded from the present analysis. This distinction is important because the selected subset was extracted from an existing experimental campaign rather than generated through a newly executed benchmark schedule.

3.3 GPU Processing Paths and Measurement Boundaries

To determine whether GPU acceleration provides a practical processing benefit, the methodology distinguishes computational execution from the additional work required to move data through the accelerator.

Three timing boundaries are evaluated.

1. **CPU AES-NI transformation.** Measures the host encryption call using the hardware-accelerated CPU library path. The interval includes output allocation performed by the encryption call but excludes cipher construction and key setup.

2. **GPU resident execution.** Measures a single CUDA kernel launch operating on data already present in device memory. The interval is measured using CUDA events and excludes host-to-device and device-to-host transfers.

3. **Prepared-buffer GPU pipeline.** Measures the synchronous host-driven processing sequence consisting of host-to-device transfer, CUDA kernel execution, device-to-host transfer, and the synchronization required to complete these operations.

CPU execution and the GPU pipeline are timed using Python's monotonic high-resolution `time.perf_counter_ns()` interface. GPU resident execution uses CUDA event timing with the required synchronization, following CUDA and CuPy measurement guidance [5, 20].

The prepared-buffer pipeline uses pinned host memory and preallocated device buffers. These resources are prepared before the timed interval. The measured GPU processing sequence can be represented conceptually as

$$T_{\text{pipeline}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + T_{\text{overhead}}, \tag{3}$$

where H2D and D2H denote the host-to-device and device-to-host stages, respectively, and the final term represents additional costs within the measured execution boundary. This expression illustrates the relevant processing stages rather than a decomposition calculated from separately aggregated measurements.

The pipeline excludes GPU compilation, memory allocation and pinning, initial host-side input staging, key preparation, and application-level input/output. It does not implement overlapping transfers and computation through multiple asynchronous CUDA streams. Consequently, its timing is broader than device-resident kernel timing, but it does not represent the complete execution of a telecommunications application.

The distinction is particularly important for assessing potential acceleration. A GPU kernel may achieve higher computational throughput while the additional transfer and synchronization costs reduce or eliminate the advantage when a complete buffer must move between host and device. Conversely, applications in which input data are already available in device memory may have different processing costs. The study measures these execution scenarios separately rather than treating kernel throughput as a substitute for broader processing performance. Recorded transfer-component intervals include synchronization effects. They therefore cannot be interpreted as isolated measurements of physical PCIe transmission time.

3.4 Performance Metrics and Statistical Analysis

The principal performance metrics are elapsed processing time, throughput, and the relative acceleration achieved by the selected GPU execution path.

For an input of N bytes and elapsed time $T_{s,r}$ measured in seconds during repetition r of session s, throughput is

$$R_{s,r} = \frac{N}{10^6 T_{s,r}}, \tag{4}$$

where throughput is expressed in decimal megabytes per second (MB/s). Input sizes are reported using binary KiB and MiB units.

To account for repeated observations and session-level variation, the analysis uses hierarchical aggregation.

First, the median elapsed time and throughput are calculated independently within each session. The principal aggregate statistics are then calculated across the five session estimates:

$$\widetilde{T} = \text{median}_s \left(\text{median}_r \, T_{s,r} \right), \qquad \widetilde{R} = \text{median}_s \left(\text{median}_r \, R_{s,r} \right). \tag{5}$$

Throughput is summarized directly from the recorded throughput observations rather than calculated by inverting the aggregated latency statistic. The acceleration factor is based on the ratio of corresponding session-level timing medians:

$$S_s = \frac{\widetilde{T}_{\text{AES-NI},s}}{\widetilde{T}_{\text{GPU},s}}, \qquad \widetilde{S} = \text{median}_{s=1,\ldots,5}(S_s). \tag{6}$$

A value greater than one indicates that the measured GPU path completes the corresponding operation faster than the host AES-NI reference. A value below one indicates that the CPU reference is faster.

The reported acceleration factor is the median of paired session-level timing ratios, not the ratio of independently aggregated CPU and GPU throughput values. The study additionally reports session ranges, interquartile ranges, coefficients of variation, and the number of sessions supporting a particular performance advantage.

An observed advantage in all five sessions is reported as $5/5$ directional consistency. This indicates repeatability within the recorded sessions but does not constitute a formal statistical significance test. The twenty repetitions within each session are not treated as twenty independent experimental sessions. Measurements are not excluded merely because they appear extreme.

This approach follows established recommendations concerning the influence of experimental conditions and hierarchical variability on computer-system performance measurements [15–17].

4 Results

The analysis distinguishes improvements in device-resident computation from those observed in the prepared-buffer GPU pipeline. The principal interest is the processing of larger encryption buffers, which provides an experimental basis for investigating GPU-assisted bulk cryptographic workloads. Smaller inputs are retained to establish how the observed advantage changes with workload size.

4.1 GPU Acceleration Across Workload Sizes

Figure 1 presents the measured GPU acceleration across the five evaluated input sizes, distinguishing AES-128 ECB and CTR execution and the two GPU measurement boundaries.

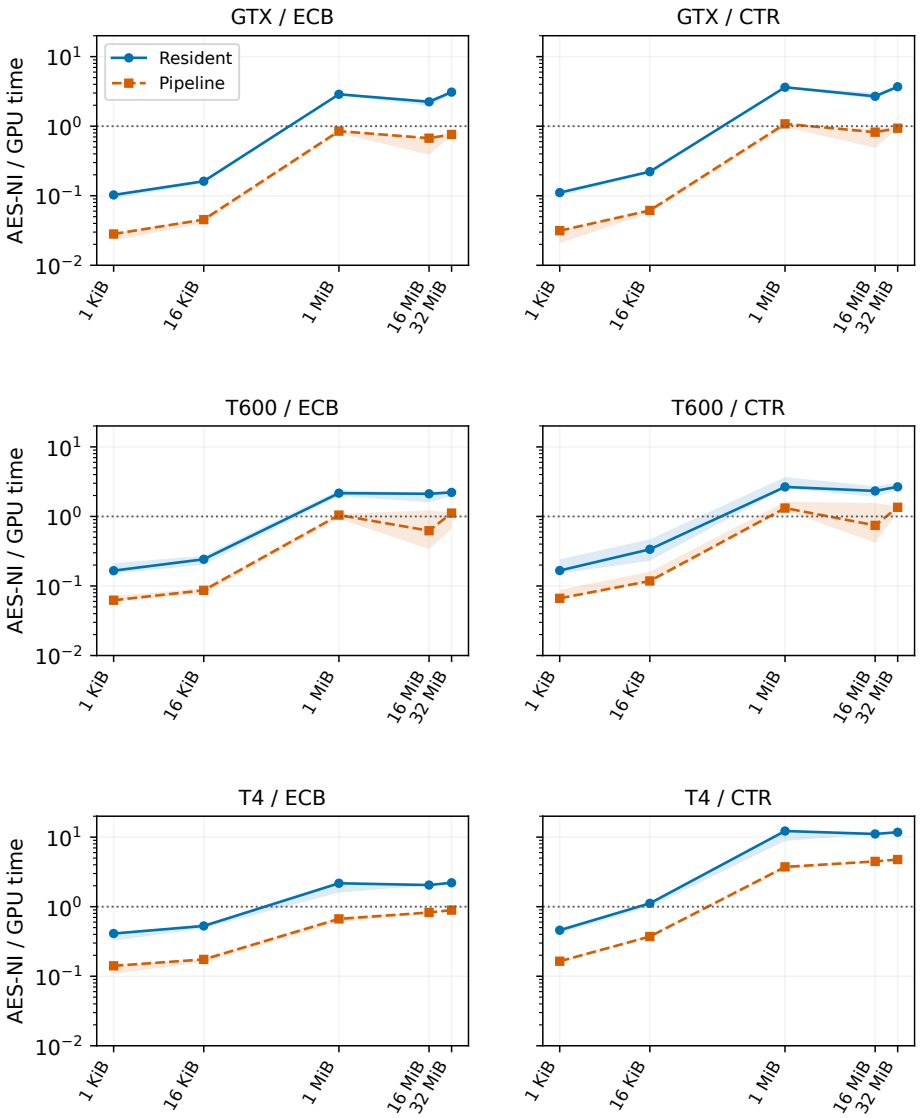


Fig. 1 — GPU-assisted AES-128 acceleration across input sizes and encryption modes. Curves represent median paired speedups relative to the corresponding host AES-NI implementation. Resident execution measures device-side computation, whereas the prepared-buffer pipeline includes synchronous host–device transfers and synchronization. Shaded regions represent the observed ranges across five study sessions, not confidence intervals. The horizontal reference denotes parity.

The results demonstrate that the benefit of GPU acceleration depends on the amount of work processed and the execution boundary used to evaluate the workload. For the 32 MiB input, resident GPU execution exceeds the corresponding CPU AES-NI reference on all three systems, in both encryption modes and in all five sessions per configuration.

This finding establishes a consistent computational advantage for the evaluated CUDA implementation when the input is already available in device memory. However, resident performance represents only the encryption kernel. Its advantage does not automatically establish faster processing when the complete buffer must be transferred from host memory to the GPU and returned after encryption.

The prepared-buffer measurements provide a more demanding test of practical acceleration because they include the synchronous transfer and execution sequence. Under this broader measurement boundary, the observed results differ substantially among systems and encryption modes. Some configurations retain a measurable advantage, while others experience a reversal in favor of the CPU AES-NI path.

Consequently, the experiments identify GPU acceleration as a conditional processing opportunity rather than a universally beneficial execution strategy.

4.2 Experimentally Confirmed Bulk Encryption Improvement

The most relevant positive results are obtained for AES-128 CTR processing of a 32 MiB buffer.

Table 2 summarizes the corresponding paired acceleration factors for resident and prepared-buffer pipeline execution, including the observed session ranges and the number of sessions in which the GPU pipeline is faster.

Table 2
Paired speedup over AES-NI at 32 MiB. R: resident; P: pipeline. Brackets show the five-session pipeline range; the last column counts P gains.

System	Mode	R	P	P range	P > 1
GTX	ECB	3.091	0.761	[0.722, 0.818]	0/5
GTX	CTR	3.693	0.935	[0.918, 1.008]	1/5
T600	ECB	2.215	1.118	[0.659, 1.144]	3/5
T600	CTR	2.666	1.354	[1.129, 1.391]	5/5
T4	ECB	2.207	0.893	[0.868, 0.901]	0/5
T4	CTR	11.758	4.761	[4.691, 5.021]	5/5

The T600 configuration provides a particularly relevant result for GPU-assisted bulk encryption on accessible hardware.

At 32 MiB, its CUDA CTR pipeline achieves a median paired acceleration of 1.354× relative to the host AES-NI implementation. The observed range extends from 1.129× to 1.391×, with the GPU completing the measured processing sequence faster in all five sessions.

The result corresponds to approximately 26.14% lower processing latency when expressed through the median of the paired CPU-to-GPU latency ratios.

Importantly, this improvement is not restricted to device-resident execution. It remains observable when the defined processing interval includes host-to-device transfer, CUDA kernel execution, device-to-host transfer, and the synchronization required for completion.

The measured result therefore establishes that a 4 GB laptop GPU can provide a repeatable advantage for a selected bulk AES-CTR workload even when data movement is included in the evaluated processing sequence.

The Tesla T4 demonstrates a larger advantage under the same workload definition. For 32 MiB CTR encryption, its prepared-buffer CUDA pipeline achieves a median paired acceleration of 4.761×, with values ranging from approximately 4.691× to 5.021× across the five sessions.

The GPU pipeline outperforms the selected host AES-NI implementation in every recorded session. These findings provide direct experimental evidence that GPU-assisted bulk AES-CTR processing can deliver measurable improvements over a hardware-accelerated CPU implementation under specific operating conditions.

However, the T600 and T4 speedup factors refer to different host processors. They must not be interpreted as a direct measurement of relative GPU performance between the two systems. The results demonstrate the feasibility of the measured acceleration on the corresponding complete hardware–software configurations.

4.3 Conditions Limiting the Acceleration Benefit

The positive CTR results do not extend uniformly across the tested configurations.

On the GTX system, the prepared-buffer AES-128 CTR pipeline achieves a median paired speedup of 0.935× at 32 MiB. The observed session range is 0.918–1.008×, with the GPU pipeline outperforming AES-NI in only one of five sessions. Although resident GPU execution is consistently faster, the broader measured pipeline does not demonstrate a repeatable improvement over the CPU reference. The GTX ECB pipeline also remains slower than AES-NI in all five sessions at the same workload size.

These findings are important because they show that the computational capability of the accelerator is not sufficient to guarantee an advantage for bulk

processing when the application requires host–device data movement. The T4 results demonstrate a different limitation associated with cipher mode. At 32 MiB, the T4 prepared-buffer CTR pipeline achieves a substantial advantage, whereas the ECB pipeline reaches only 0.893× relative to the corresponding CPU AES-NI path.

The ECB pipeline is slower in every recorded session despite the resident GPU execution remaining faster than AES-NI. The T600 also exhibits mode-dependent behavior. Its 32 MiB ECB pipeline has a median speedup above one, but the GPU is faster in only three of five sessions. Thus, a positive aggregate median does not necessarily indicate a consistently favorable processing configuration.

The experimental evidence identifies an important practical constraint: the conditions supporting GPU acceleration must be established for the specific workload and encryption mode rather than inferred solely from the presence of a parallel accelerator. For potential bulk cryptographic applications, these results indicate that processing volume alone is not sufficient to determine whether GPU execution will be advantageous.

4.4 Absolute Encryption Throughput

Relative acceleration factors are useful for determining whether a GPU processing path improves upon its host CPU reference. However, absolute throughput is also necessary to characterize the processing capacity of the accelerator itself.

At 32 MiB in CTR mode, the measured prepared-buffer pipeline throughput is approximately 1188.9 MB/s on GTX, 2049.4 MB/s on T600, and 3713.9 MB/s on T4. The corresponding device-resident throughput values are 4585.6 MB/s, 4328.9 MB/s, and 9166.8 MB/s, respectively.

These measurements demonstrate that the T4 provides the highest absolute CTR throughput among the evaluated systems under this workload. They also show that GPU pipeline throughput can differ substantially from device-resident throughput even when the same encryption implementation and input size are used. The distinction matters for system design because the amount of data that can be processed per second depends on the actual execution sequence required to make the input available and recover the result. A configuration with high resident throughput may not deliver a correspondingly high host-observed processing rate.

Absolute throughput and host-relative acceleration therefore describe different aspects of the measured processing capability. The experiments do not isolate the individual contributions of PCIe transfer time, synchronization, and runtime overhead sufficiently to attribute the entire resident-to-pipeline difference to one component.

4.5 Cipher-Mode Dependence and Host Processing Behavior

The T4 measurements reveal an additional factor affecting the interpretation of GPU-assisted acceleration. For 32 MiB inputs, the baseline GPU pipeline achieves nearly identical absolute throughput in ECB and CTR modes, approximately 3713 and 3714 MB/s, respectively. The corresponding resident GPU rates are also similar, at approximately 9177 MB/s for ECB and 9167 MB/s for CTR.

Nevertheless, the relative acceleration factors differ markedly: 0.893× for ECB and 4.761× for CTR. The principal distinction is associated with the measured host AES-NI execution rates. At 32 MiB, the T4 host's AES-NI ECB throughput is approximately 5.378× its CTR throughput. The ratio ranges from 5.250× to 5.795× across sessions. By contrast, the corresponding ratios on the GTX and T600 host configurations are approximately 1.26×. The T4 host disparity is consistent across the recorded sessions, but its underlying cause was not isolated experimentally.

Differences in cryptographic library behavior, mode-specific processing, processor characteristics, and runtime environment may contribute, but the available evidence does not identify a single causal mechanism. This observation is significant because a large GPU acceleration factor may result from the behavior of the CPU execution path as well as the absolute performance of the GPU.

Consequently, the reported CTR improvement must be interpreted relative to the corresponding hardware-accelerated CPU implementation. The result does not establish that the T4 GPU itself processes CTR several times faster than ECB. For heterogeneous cryptographic systems, the finding reinforces the need to assess the complete selected execution path rather than assume that the same relative acceleration will occur under every implementation of an encryption mode.

4.6 Workload-Size Effects and Experimental Consistency

The measured performance relationships also depend on workload size. Although larger buffers can provide more independent AES operations for GPU execution, throughput does not increase monotonically across all evaluated sizes and platforms.

On GTX, a local throughput reduction occurs at 16 MiB relative to both 1 MiB and 32 MiB. For the retained CUDA implementation, this pattern is observed across both encryption modes and both GPU timing scopes in all five study sessions. On T4, resident measurements show the corresponding local reduction in four sessions, whereas the pipeline measurements do not exhibit the same pattern.

These differences demonstrate that the observed size dependence is not a universal property of GPU AES execution. The benchmark processes workload

sizes in ascending order, which also prevents complete separation of buffer-size effects from time-dependent execution-environment behavior. The available results therefore do not establish whether the local reduction is caused by memory behavior, scheduling, device clocks, or another system-level mechanism.

Session-level consistency is particularly important when the measured acceleration approaches parity. For example, GTX CTR at 32 MiB occasionally favors the GPU, but the median result and directional count do not support a consistently positive pipeline advantage.

Similarly, T600 ECB at 32 MiB has a positive median acceleration while showing mixed results across sessions. By contrast, the T600 and T4 32 MiB CTR pipeline improvements remain positive in all five recorded sessions. For the T600 CTR configuration, the paired speedup has an interquartile range of 0.030896 and a sample coefficient of variation of approximately 8.18%.

These descriptive statistics provide information about the stability of the observed advantage within the tested sessions. They do not constitute confidence intervals or establish that the same performance improvement will occur on other devices or in different execution environments.

The results therefore distinguish repeatable acceleration under observed conditions from isolated favorable outcomes.

4.7 Summary of Experimental Findings

The experiments establish three principal findings relevant to GPU-assisted bulk AES processing.

First, device-resident CUDA execution provides a consistent computational advantage over the evaluated AES-NI CPU path at 32 MiB on all three systems and in both tested modes.

Second, this advantage can persist when host–device transfers are included. The T600 and T4 systems demonstrate repeatable prepared-buffer AES-CTR improvements at 32 MiB, confirming that GPU-assisted bulk processing can be beneficial under specific execution conditions.

Third, the improvement is conditional. The GTX pipeline does not achieve a consistently positive result, and the T4 ECB measurements demonstrate that the observed benefit also depends on cipher mode and host implementation behavior.

Together, these findings identify experimentally supported opportunities and constraints for GPU-assisted bulk AES processing. They provide a basis for assessing heterogeneous cryptographic processing strategies in telecommunications-oriented systems, while the performance of actual packet batching, authenticated encryption, and complete network processing remains outside the scope of the present measurements.

5 Discussion

The experimental findings provide evidence that GPU-assisted AES processing can improve bulk encryption performance under identifiable operating conditions. Their significance for telecommunications lies not in a new AES algorithm, but in the possibility of using existing heterogeneous computing resources to accelerate selected cryptographic workloads.

The central observation is that the benefit of parallel AES execution can survive host–device data movement. However, this improvement is conditional and depends on the encryption mode, processing volume, available CPU acceleration, and data location. These conditions must be considered when evaluating GPU-assisted encryption as a component of a larger communication system.

5.1 Implications for Bulk Cryptographic Processing

The observed AES-CTR improvements establish that GPU acceleration can be beneficial for sufficiently substantial workloads on particular hardware–software configurations. The positive prepared-buffer results are especially relevant because they demonstrate an advantage beyond isolated CUDA kernel execution.

For telecommunications systems, the main opportunity is the processing of substantial volumes of independent cryptographic work. Network gateways, communication servers, and data-processing services may encounter workloads in which multiple independent data units can be organized for parallel execution. AES-CTR is suitable for this form of processing because distinct counter blocks can be encrypted independently, provided that the counter and nonce requirements are satisfied [3]. GPU execution can exploit this independence by distributing AES transformations across many processing threads.

The measured improvement on the T600 configuration is relevant because it demonstrates that a repeatable bulk-processing advantage is not restricted to the tested datacenter-oriented configuration. Nevertheless, the results apply to the particular host CPU, GPU implementation, workload, and prepared-buffer execution boundary used in the experiment. The findings do not identify a universal minimum batch size. Although a 32 MiB workload produces a consistent advantage on selected systems, the experiments do not establish that this size is optimal or that larger workloads will necessarily improve performance further. More importantly, a contiguous encryption buffer is not equivalent to a group of independently managed network packets. Organizing packet data into a GPU workload requires additional processing whose performance was not evaluated.

Thus, the direct experimental contribution is the identification of favorable bulk-processing conditions. Their translation into packet-oriented workloads remains an architectural opportunity rather than a demonstrated deployment result.

5.2 Opportunities for Telecommunications Architectures

The findings suggest several ways in which GPU resources could complement existing CPU cryptographic acceleration in heterogeneous telecommunications systems.

One possibility is bulk processing within communication gateways or server-side data-protection services. Where a system already handles substantial contiguous buffers, GPU-assisted AES may offer an additional processing path. The measured prepared-buffer results indicate that such an approach warrants consideration even when the host CPU supports AES-NI.

A second possibility is the aggregation of independent encryption requests into larger GPU workloads. This approach could increase the amount of parallel work available for each accelerator invocation and distribute execution overhead across multiple data units.

However, batching introduces a trade-off between throughput and latency. Waiting for sufficient work to form a batch can increase queueing delay, particularly during periods of low traffic. Packet metadata, independent cryptographic contexts, counter allocation, and result ordering also introduce requirements absent from the contiguous-buffer experiments. Consequently, the results support investigating batching for throughput-oriented workloads, but they do not establish an optimal batching policy or a guaranteed improvement in packet-processing performance.

A third opportunity concerns systems in which data are already processed on the GPU. If encryption can operate on device-resident buffers without additional host transfers, the relevant execution boundary differs from the synchronous host-to-host pipeline evaluated in this study.

The consistently favorable resident measurements provide evidence of the computational potential of this arrangement. Nevertheless, a practical device-resident application would still need to account for the acquisition, organization, authentication, and delivery of the protected data.

These opportunities point toward a workload-aware architecture in which CPU AES-NI remains available for smaller or latency-sensitive operations, while GPU processing is considered for suitable bulk workloads. Such an architecture is a design implication of the measurements. It was not implemented or experimentally evaluated as part of the present study.

5.3 Relationship to IPsec and Authenticated Encryption

AES-CTR is specified as a confidentiality mechanism for IPsec Encapsulating Security Payload (ESP) in RFC 3686 [21]. AES-GCM, which combines counter-mode encryption with authentication, is specified for IPsec ESP in RFC 4106 [22].

These standards establish a direct connection between parallelizable AES processing and standardized telecommunications security mechanisms. In particular, the independent generation of counter-mode keystream blocks provides opportunities for GPU-based parallel execution. However, the present experiments evaluate AES-128 CTR encryption rather than complete IPsec ESP or AES-GCM processing. The measured acceleration therefore characterizes the encryption component under the defined experimental conditions, not the performance of an authenticated network-security protocol.

An IPsec implementation must account for operations that are not represented by the measured AES-CTR processing interval. These include packet framing, security-association state, nonce and sequence-number management, authentication, and the processing required to deliver protected packets. For AES-GCM, the additional GHASH computation and authentication-tag processing must also be considered. A measured improvement in the counter-mode encryption component does not determine the performance of the complete authenticated-encryption operation.

The findings are therefore relevant to evaluating the encryption component of possible GPU-assisted IPsec architectures, but they do not demonstrate that an IPsec tunnel or AES-GCM implementation would achieve the same acceleration factors.

A practical extension would need to preserve the security requirements of the selected protocol while measuring throughput and latency across the complete packet-processing path. In particular, independent packet contexts must not be combined in a way that compromises counter uniqueness, authentication boundaries, or the association between encrypted data and its protocol metadata.

5.4 Processing Constraints and Execution-Path Selection

The experiments identify several constraints that are relevant to the selection of a cryptographic execution path.

The first is data location. Device-resident execution avoids host–device transfers within the measured interval, whereas the prepared-buffer pipeline includes the movement required to process a host-resident buffer. The different outcomes observed under these boundaries show that the computational benefit of GPU execution can be reduced or eliminated by additional processing operations.

The second constraint is the existing performance of the host CPU. Hardware-assisted AES-NI can provide substantial encryption throughput, and a GPU advantage must be established against the available host implementation rather than assumed from an unaccelerated software reference.

The third constraint is encryption-mode behavior. The T4 results demonstrate

that similar absolute GPU throughput can produce different relative acceleration outcomes when the corresponding CPU modes have substantially different execution rates. The underlying cause of this host behavior was not isolated. Accordingly, the findings do not support attributing the observed difference to a particular processor feature, library version, or cloud mechanism.

The fourth constraint is workload size. GPU execution introduces fixed and variable costs that may be less favorable for smaller processing requests. The experimental size grid also exhibits non-monotonic behavior, preventing the identification of a simple universal CPU-to-GPU crossover threshold.

These constraints suggest that a deployment-oriented execution policy would need to consider workload characteristics and data location rather than selecting an accelerator solely on the basis of peak kernel throughput. An adaptive selection mechanism could potentially assign suitable bulk workloads to the GPU while retaining CPU execution for other requests. Determining an effective policy would require workload-specific measurements and remains outside the demonstrated results.

5.5 Experimental Validity and Generalization

The observed advantages are supported by repeated measurements under the tested conditions, but several factors limit their generalization. Each configuration represents one host or cloud allocation with five study sessions. These sessions provide evidence of repeatability within the selected environment, not a statistical characterization of the wider population of comparable machines.

The configurations differ in processor models, GPU devices, operating systems, drivers, and supporting software. Their performance differences therefore cannot be attributed causally to a single hardware or software characteristic. The CPU reference is a selected AES-NI-enabled library path rather than a comprehensive evaluation of the fastest possible multithreaded CPU encryption service. Similarly, the CUDA kernel is a validated implementation, not a demonstrated performance limit of the GPUs.

The prepared-buffer pipeline excludes initial host staging, allocation, authentication, external input/output, and application-level orchestration. Its results must therefore not be interpreted as complete telecommunications-system throughput. The fixed ascending order of input sizes also limits causal interpretation of the observed non-monotonic performance behavior. In particular, the local 16 MiB reduction cannot be assigned to memory effects, GPU scheduling, processor clocks, or thermal conditions without additional evidence.

Finally, the use of five-session ranges and directional consistency does not establish formal statistical significance. The results identify observed repeatable

conditions rather than guaranteed future performance.

These limitations define the boundary of the contribution: validated evidence of bulk AES processing improvements under measured conditions, not a universal claim about GPU cryptographic acceleration.

5.6 Future Research Directions

The most direct extension is the evaluation of GPU-assisted AES processing within a representative telecommunications workload, particularly a communication gateway or bulk data-protection service.

Such an investigation could determine whether the observed AES-CTR improvements remain beneficial when packet aggregation, cryptographic context management, and output delivery are included.

A second direction concerns authenticated encryption. A GPU implementation of AES-GCM, including GHASH, authentication-tag processing, and the required security state, would provide a more direct basis for evaluating GPU acceleration in IPsec-oriented systems.

A third direction is the investigation of workload-aware CPU–GPU scheduling. Such a system could evaluate whether processing volume, data residency, and latency requirements justify execution on the GPU or the host CPU.

A fourth direction involves transfer-aware processing techniques, including overlapping communication and computation, persistent device-resident data, and supported direct-data-transfer mechanisms. These approaches could change the practical cost of cryptographic offloading, but their performance cannot be predicted from the synchronous measurements alone.

Finally, additional independent systems and more densely sampled workloads would help establish the external validity of the observed acceleration conditions.

These extensions are not prerequisites for the present empirical contribution. Rather, they identify the necessary steps for translating measured bulk AES acceleration into deployable cryptographic services for telecommunications infrastructure.

6 Conclusion

This study demonstrates that GPU-assisted AES encryption can improve bulk cryptographic processing performance under specific operating conditions, even when the host processor already supports hardware-accelerated AES-NI. The contribution is the experimental identification of conditions under which this improvement remains observable after including synchronous host–device transfers, together with the practical constraints that determine its potential relevance to

telecommunications systems.

The strongest evidence is obtained for AES-128 CTR encryption of 32 MiB buffers. The validated CUDA pipeline achieved a median paired acceleration of 1.354× on the T600 configuration and 4.761× on the T4 configuration, relative to their respective host AES-NI implementations. Both improvements were observed in all five experimental sessions. The T600 result is particularly relevant because it establishes that a repeatable bulk-processing advantage is possible on the tested 4 GB laptop GPU, without relying on a datacenter-class accelerator.

However, the experiments also establish important boundaries. The GTX configuration did not provide a consistent transfer-inclusive improvement over AES-NI, while the T4 ECB pipeline remained slower than its corresponding CPU reference. Although device-resident GPU execution consistently outperformed AES-NI at 32 MiB, the advantage did not always survive the inclusion of data movement and synchronization. The results therefore identify workload-dependent acceleration opportunities rather than a universal GPU performance advantage.

For telecommunications infrastructure, these findings support further consideration of GPU-assisted bulk encryption, particularly in systems capable of processing large buffers or organizing independent cryptographic operations into suitable workloads. AES-CTR parallelism provides a relevant computational basis for such architectures, including potential acceleration of the encryption component within IPsec-oriented processing systems. Nevertheless, the experiments do not establish improvements in complete IPsec ESP, AES-GCM, or packet-processing pipelines, where authentication, protocol state, batching, and communication latency introduce additional requirements.

The validation-first experimental design, paired measurements, explicit timing boundaries, and preserved session-level evidence support the reproducibility and interpretation of the reported findings. The principal implication is that GPU-assisted AES should be investigated as a targeted processing strategy for suitable bulk cryptographic workloads, rather than as a direct replacement for CPU-based encryption. The findings provide experimentally grounded operating conditions and practical constraints that can inform the development of heterogeneous telecommunications architectures, while identifying authenticated encryption, packet aggregation, and workload-aware execution as areas requiring application-level evaluation.

References

[1] National Institute of Standards and Technology. Advanced encryption standard (aes). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards

and Technology, 2023.

[2] Intel Corporation. Intel advanced encryption standard instructions (aes-ni), 2012. URL https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[3] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001.

[4] Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate cpu vs. gpu performance without the answer. In *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 134–144, 2011. doi: 10.1109/ISPASS.2011.5762730.

[5] NVIDIA Corporation. Cuda c++ best practices guide, n.d. URL https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Accessed 2026.

[6] Svetlin A. Manavski. Cuda compatible gpu as an efficient hardware accelerator for aes cryptography. In *Proceedings of the IEEE International Conference on Signal Processing and Communications*, 2007. doi: 10.1109/ICSPC.2007.4728256.

[7] Owen Harrison and John Waldron. Aes encryption and decryption on the gpu. In *GPU Gems 3*, chapter 36. Addison-Wesley Professional, 2007. URL https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-36-aes-encryption-and-decryption-gpu.

[8] Keisuke Iwai, Naoki Nishikawa, and Takakazu Kurokawa. Acceleration of aes encryption on cuda gpu. *International Journal of Networking and Computing*, 2(1):131–145, 2012. doi: 10.15803/ijnc.2.1_131.

[9] Canhui Wang and Xiaowen Chu. Gpu accelerated aes algorithm, 2019. URL https://arxiv.org/abs/1902.05234.

[10] Cihangir Tezcan. Optimization of advanced encryption standard on graphics processing units. *IEEE Access*, 9:67315–67326, 2021. doi: 10.1109/ACCESS.2021.3077551.

[11] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.

[12] Wai-Kong Lee, Seog Chung Seo, Hwajeong Seo, Dong Cheon Kim, and Seong Oun Hwang. Speed record of aes-ctr and aes-ecb bit-sliced implementation on gpus. *IEEE Embedded Systems Letters*, 16(4):481–484, 2024. doi: 10.1109/LES.2024.3409725.

[13] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100x gpu vs. cpu myth: An evaluation of throughput computing on cpu and gpu. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, pages 451–460, 2010. doi: 10.1145/1815961.1816021.

[14] Mark Harris. How to overlap data transfers in cuda c/c++. NVIDIA Technical Blog, 2012. URL https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/.

[15] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[16] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time.

In *Proceedings of the International Symposium on Memory Management*, 2013. doi: 10.1145/2464157.2464160.

[17] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[18] PyCryptodome. Pycryptodome documentation, n.d. URL `https://pycryptodome.readthedocs.io/`. Accessed 2026.

[19] CuPy. cupy.rawkernel. `https://docs.cupy.dev/en/stable/reference/generated/cupy.RawKernel.html`, n.d.. Accessed 2026.

[20] CuPy. Performance best practices. `https://docs.cupy.dev/en/stable/user_guide/performance.html`, n.d.. Accessed 2026.

[21] Russ Housley. Using advanced encryption standard (AES) counter mode with IPsec encapsulating security payload (ESP). RFC 3686, Internet Engineering Task Force, January 2004. URL `https://www.rfc-editor.org/info/rfc3686`.

[22] John Viega and David A. McGrew. The use of galois/counter mode (GCM) in IPsec encapsulating security payload (ESP). RFC 4106, Internet Engineering Task Force, June 2005. URL `https://www.rfc-editor.org/info/rfc4106`.